

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY COURSE
CODE: CSA5115

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

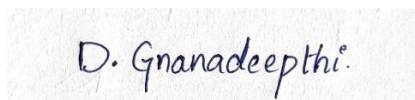
Total Marks: 50

Annexure A

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB). C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures. B. Refactor the code to minimize redundant computations and memory usage. C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.

B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).

C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic. B. Optimize exponentiation operations for large prime values.

C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic. B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

ANSWERS

2. Performance Analysis of Block Cipher Modes of Operation

A. Debug Incorrect Padding and IV Handling Causing Decryption Failures

The decryption failures are mainly caused by incorrect padding and improper Initialization Vector (IV) handling.

Padding Issues

- Ensure the plaintext is padded to the AES block size (16 bytes) using a standard padding scheme such as **PKCS#7**.
- During decryption, remove the padding only after verifying that it is valid.
- Incorrect or inconsistent padding leads to decryption errors or corrupted plaintext.

IV Handling Issues

- ECB mode does **not** use an IV.
- CBC, CFB, and OFB modes require a **unique, random IV** for each encryption.
- The same IV used during encryption must be provided during decryption.
- Reusing or modifying the IV causes incorrect decryption and weakens security.

B. Refactor the Code to Minimize Redundant Computations and Memory Usage

To improve performance:

- Reuse the AES key schedule instead of generating round keys repeatedly.
- Process data **block by block** instead of loading the entire file into memory.
- Avoid unnecessary copying of arrays or buffers.
- Reuse allocated memory wherever possible.
- Store only the current block and IV instead of all intermediate values.
- Use optimized cryptographic libraries for efficient encryption and decryption.

These improvements reduce both execution time and memory consumption.

C. Compare and Optimize the Implementation

Mode	IV Required	Error Propagation	Performance	Security	Suitable for Real-Time
ECB	No	No	Very Fast	Poor	No
CBC	Yes	High	Moderate	High	Moderate
CFB	Yes	Low	Moderate	High	Yes
OFB	Yes	No	Fast	High	Yes

Analysis

- **ECB** is fast but reveals plaintext patterns and is not recommended.
- **CBC** provides strong security but has higher latency due to block dependency.
- **CFB** supports stream-like encryption with good security and moderate performance.
- **OFB** generates a keystream independent of plaintext, resulting in low latency, no error propagation, and efficient performance.

3. Debugging RSA Cryptosystem Implementation

A. Identify Issues in Prime Generation, Key Generation, and Modular Exponentiation

Prime Generation

- Small or non-random primes reduce security.
- Failure to verify primality may produce invalid keys.
- Use probabilistic primality tests such as **Miller-Rabin**.

Key Generation

- Ensure that **p** and **q** are distinct prime numbers.
- Compute:

$$n=p \times q$$

$$\phi(n)=(p-1)(q-1)$$

Choose e such that:

$$\gcd(e, \phi(n)) = 1$$

Compute the private key:

$$D = e^{-1} \pmod{\phi(n)}$$

Incorrect selection of e or d results in decryption errors.

Modular Exponentiation

Direct computation of large powers causes overflow and poor performance.

Use fast modular exponentiation (Square-and-Multiply) for efficient calculations.

B. Optimize the Algorithm

Fast Modular Exponentiation

Computes:

$$a^b \bmod n$$

efficiently using repeated squaring.

- Reduces time complexity from $O(b)$ to $O(\log b)$.

Chinese Remainder Theorem (CRT)

Instead of computing:

$$M = C^d \pmod{n}$$

compute separately:

$$M_p = C^d \pmod{p}$$

$$M_q = C^d \pmod{q}$$

and combine the results.

Advantages

- Faster decryption (approximately 3–4 times).
- Reduced computational cost.
- Improved overall performance.

C. Evaluate the Trade-offs Between Key Size, Security, and Performance

Key Size	Security	Encryption Speed	Decryption Speed	Suitable For
1024-bit	Moderate	Fast	Fast	Legacy systems
2048-bit	High	Moderate	Moderate	Standard applications
4096-bit	Very High	Slow	Slow	High-security environments

Analysis

- Smaller keys provide faster computation but lower security.
- Larger keys offer stronger security but increase computation time and memory usage.
- 2048-bit RSA is widely recommended because it balances security and performance effectively.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysiss	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation